

Zomato Data Analysis Using Python

Understanding customer preferences and restaurant trends is important for making informed business decisions in food industry. In this article, we will analyze Zomato's restaurant dataset using Python to find meaningful insights. We aim to answer questions such as:

- Do more restaurants provide online delivery compared to offline services?
- Which types of restaurants are most favored by the general public?
- What price range do couples prefer for dining out?

Step 1: Importing necessary Python libraries.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 2: Creating the data frame.

```
[3]: dataframe = pd.read_csv("Zomato-data-.csv")
print(dataframe.head())
```

	name	online_order	book_table	rate	votes	\
0	Jalsa	Yes	Yes	4.1/5	775	
1	Spice Elephant	Yes	No	4.1/5	787	
2	San Churro Cafe	Yes	No	3.8/5	918	
3	Addhuri Udupi Bhojana	No	No	3.7/5	88	
4	Grand Village	No	No	3.8/5	166	

	approx_cost(for two people)	listed_in(type)
0	800	Buffet
1	800	Buffet
2	800	Buffet
3	300	Buffet
4	600	Buffet

Step 3: Data Cleaning and Preparation

Before moving further we need to clean and process the data.

1. Convert the rate column to a float by removing denominator characters.

- **dataframe['rate']=dataframe['rate'].apply(handleRate):** Applies the **handleRate** function to clean and convert each rating value in the 'rate' column.

```
[4]: def handleRate(value):
      value=str(value).split('/')
      value=value[0];
      return float(value)

dataframe['rate']=dataframe['rate'].apply(handleRate)
print(dataframe.head())
```

	name	online_order	book_table	rate	votes	\
0	Jalsa	Yes	Yes	4.1	775	
1	Spice Elephant	Yes	No	4.1	787	
2	San Churro Cafe	Yes	No	3.8	918	
3	Addhuri Udupi Bhojana	No	No	3.7	88	
4	Grand Village	No	No	3.8	166	

	approx_cost(for two people)	listed_in(type)
0	800	Buffet
1	800	Buffet
2	800	Buffet
3	300	Buffet
4	600	Buffet

Step 4: Exploring Restaurant Types

1. Let's see the **listed_in (type)** column to identify popular restaurant categories.

```

colors = ["r", "g", "y", "b"]

sns.countplot(x=dataframe['listed_in(type)'], palette=colors)
plt.xlabel("Type of Restaurant")
plt.ylabel("Count")
plt.title("Distribution of Restaurant Types")
plt.show()

```

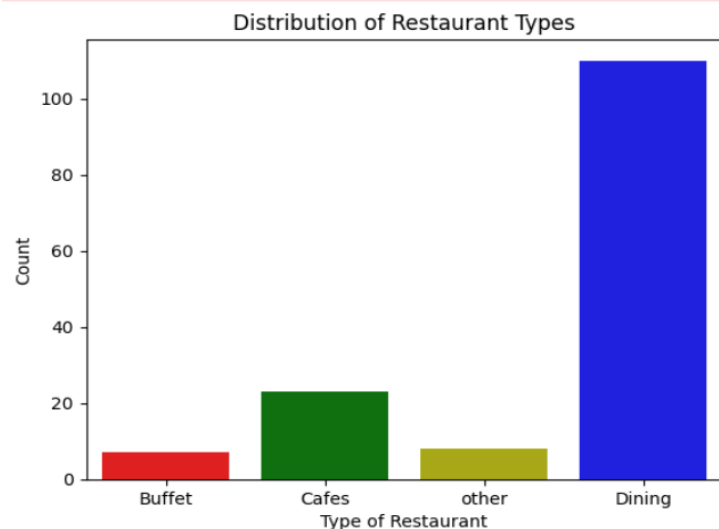
C:\Users\sarve\AppData\Local\Temp\ipykernel_24720\91021357.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```

sns.countplot(x=dataframe['listed_in(type)'], palette=colors)

```



2. Votes by Restaurant Type

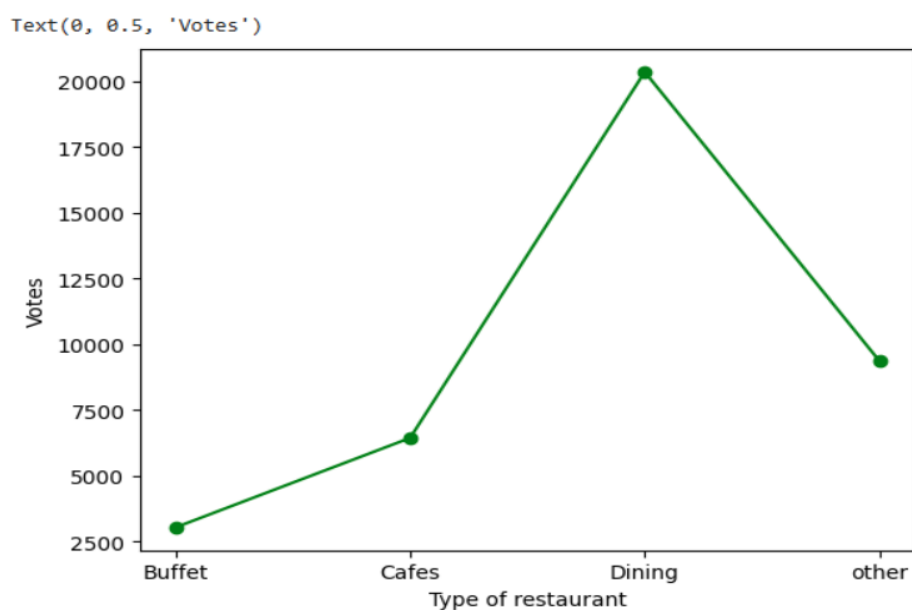
Here we get the count of votes for each category.

```

1 grouped_data = dataframe.groupby('listed_in(type)')['votes'].sum()
2 result = pd.DataFrame({'votes': grouped_data})
3 plt.plot(result, c='green', marker='o')
4 plt.xlabel('Type of restaurant')
5 plt.ylabel('Votes')

```

Output:



Step 6: Online Order Availability

Exploring the **online_order** column to see how many restaurants accept online orders.

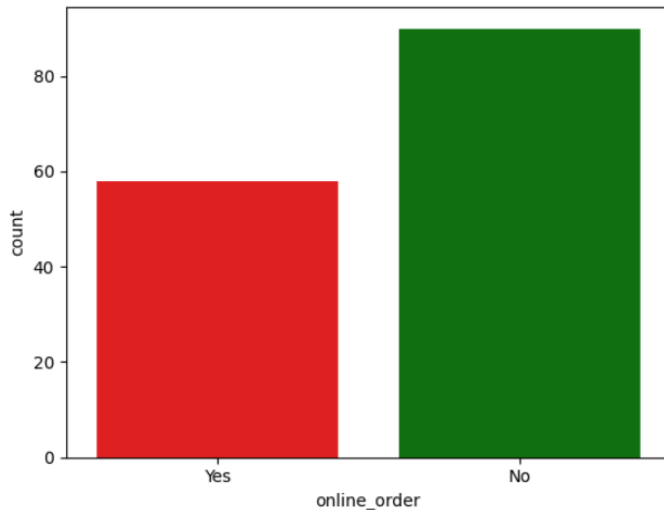
```
[13]: colors = ["r", "g"]  
sns.countplot(x=dataframe['online_order'], palette=colors)
```

C:\Users\sarve\AppData\Local\Temp\ipykernel_24720\3518737076.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x=dataframe['online_order'], palette=colors)
```

```
[13]: <Axes: xlabel='online_order', ylabel='count'>
```

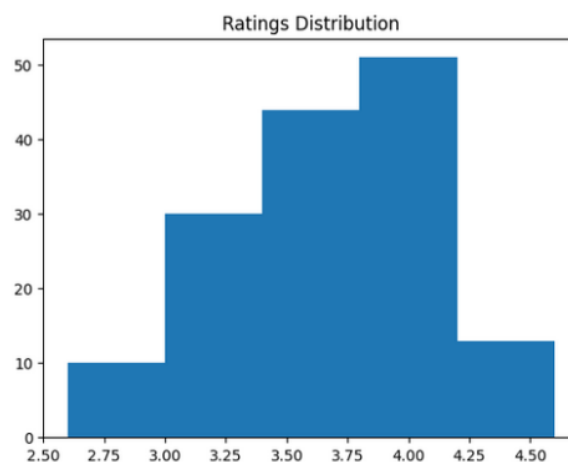


Step 7: Analyze Ratings

Checking the distribution of ratings from the **rate** column.

```
plt.hist(dataframe['rate'], bins=5)  
plt.title('Ratings Distribution')  
plt.show()
```

Output:

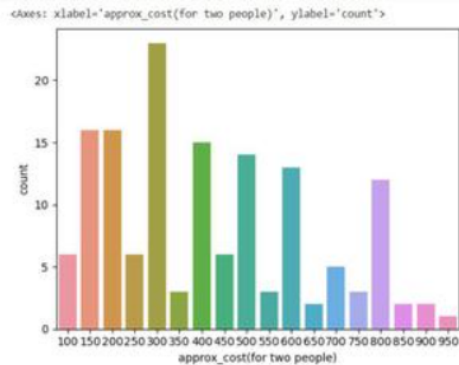


Step 8: Approximate Cost for Couples

Analyze the `approx_cost(for two people)` column to find the preferred price range.

```
couple_data=dataframe['approx_cost(for two people)']  
sns.countplot(x=couple_data)
```

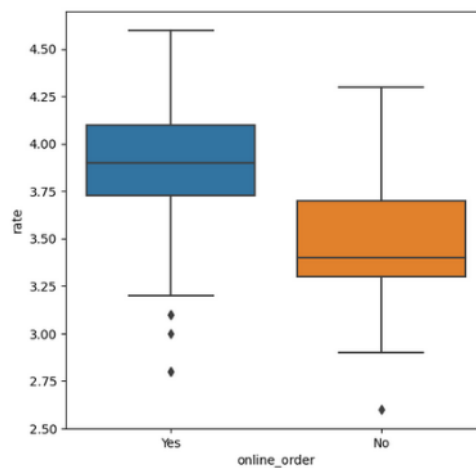
Output:



Conclusion: The majority of couples prefer restaurants with an approximate cost of 300 rupees.

```
plt.figure(figsize = (6,6))  
sns.boxplot(x = 'online_order', y = 'rate', data = dataframe)
```

Output:



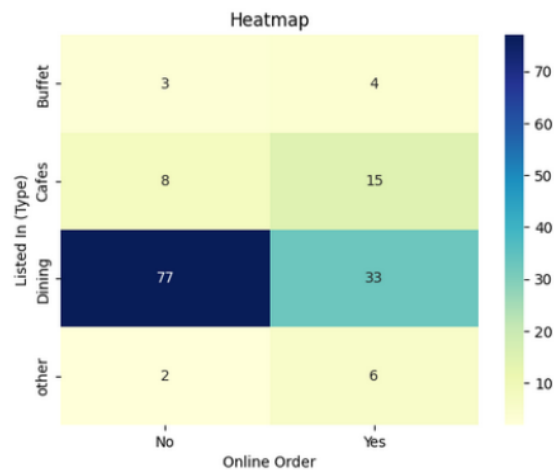
Conclusion: Offline orders received lower ratings in comparison to online orders which obtained excellent ratings.

```

pivot_table = dataframe.pivot_table(index='listed_in(type)', columns='online_order',
aggfunc='size', fill_value=0)
sns.heatmap(pivot_table, annot=True, cmap='YlGnBu', fmt='d')
plt.title('Heatmap')
plt.xlabel('Online Order')
plt.ylabel('Listed In (Type)')
plt.show()

```

Output:



With this we can say that dining restaurants primarily accept offline orders whereas cafes primarily receive online orders. This suggests that clients prefer to place orders in person at restaurants but prefer